INSTITUTO FEDERAL
São Paulo
Campus Campos do Jordão

Instituto Federal de Educação, Ciência e Tecnologia de São Paulo – Campus Campos do Jordão

DISCENTES: AUDREY DE TOLEDO, BRUNO HENRIQUE, DANIELA COSTA, MATHEUS COSTA E PAULO CEZAR

INSTITUTO FEDERAL
São Paulo
Campus Campos do Jordão

SISTEMA DE AVALIAÇÃO DE EXPRESSÕES MATEMÁTICAS

Uma calculadora de terminal que avalia expressões matemáticas simples digitadas pelo usuário, mostrando o histórico de cálculos realizados na sessão.

OPERAÇÕES

1- Avaliar expressão

2-Exibir histórico

3- Buscar por resultado

4- Ordenar histórico

5- Limpar histórico

0- Sair = Encerra o programa.

INSTITUTO FEDERAL
São Paulo
Campus Campos do Jordão

ANÁLISE DE COMPLEXIDADE BIG-O
Sistema de Avaliação de Expressões Matemáticas

Este relatório analisa a complexidade computacional do sistema de avaliação de expressões matemáticas. O objetivo é medir como o tempo de execução cresce conforme aumenta o tamanho da entrada. A análise usando Big-O permite entender a eficiência dos algoritmos usados para avaliar expressões, ordenar o histórico e buscar resultados.

ANÁLISE

- **Avaliação da expressão — O(n):** as três funções (parentesesValidos, infixoParaPosfixo, avaliarPosfixaRecursiva) percorrem a expressão uma única vez cada, operando com pilha em O(1) por elemento.
- **Ordenação — O(n^2):** o Bubble Sort usa dois for aninhados, o que resulta em comparações proporcionais ao quadrado do tamanho do histórico.
- **Busca Binária — O(log n):** a cada iteração descarta metade dos elementos, crescendo de forma logarítmica.

ANÁLISE DE COMPLEXIDADE BIG-O

Sistema de Avaliação de Expressões Matemáticas

O(n)	**Avaliação da Expressão**	Shunting-Yard + Pilha Recursiva
$O(n^2)$	**Ordenação do Histórico**	Bubble Sort
O(log n)	**Busca no Histórico**	Busca Binária

ANÁLISE

- A avaliação O(n) é eficiente e escala bem — cada caractere da expressão é visitado uma vez.
- O Bubble Sort $O(n^2)$ é o gargalo do sistema; com histórico grande, ficaria lento — algoritmos como Merge Sort ou Quick Sort (O(n log n)) seriam superiores
- A Busca Binária O(log n) é excelente, mas depende do histórico estar ordenado —daí a chamada ordenarHistorico() antes de buscarExpressao() no caso 3 do código.

Tabela de Crescimento de Operações

Número de operações executadas para cada complexidade conforme n cresce:

n (entradas)	O(n) Avaliação	O(n^2) Bubble Sort	O(log n) Busca Binária
1	1	1	1
5	5	25	3
10	10	100	4
50	50	2K	6
100	100	10K	7
500	500	250K	9
1,000	1K	1.0M	10
5,000	5K	25.0M	13
10,000	10K	100.0M	14

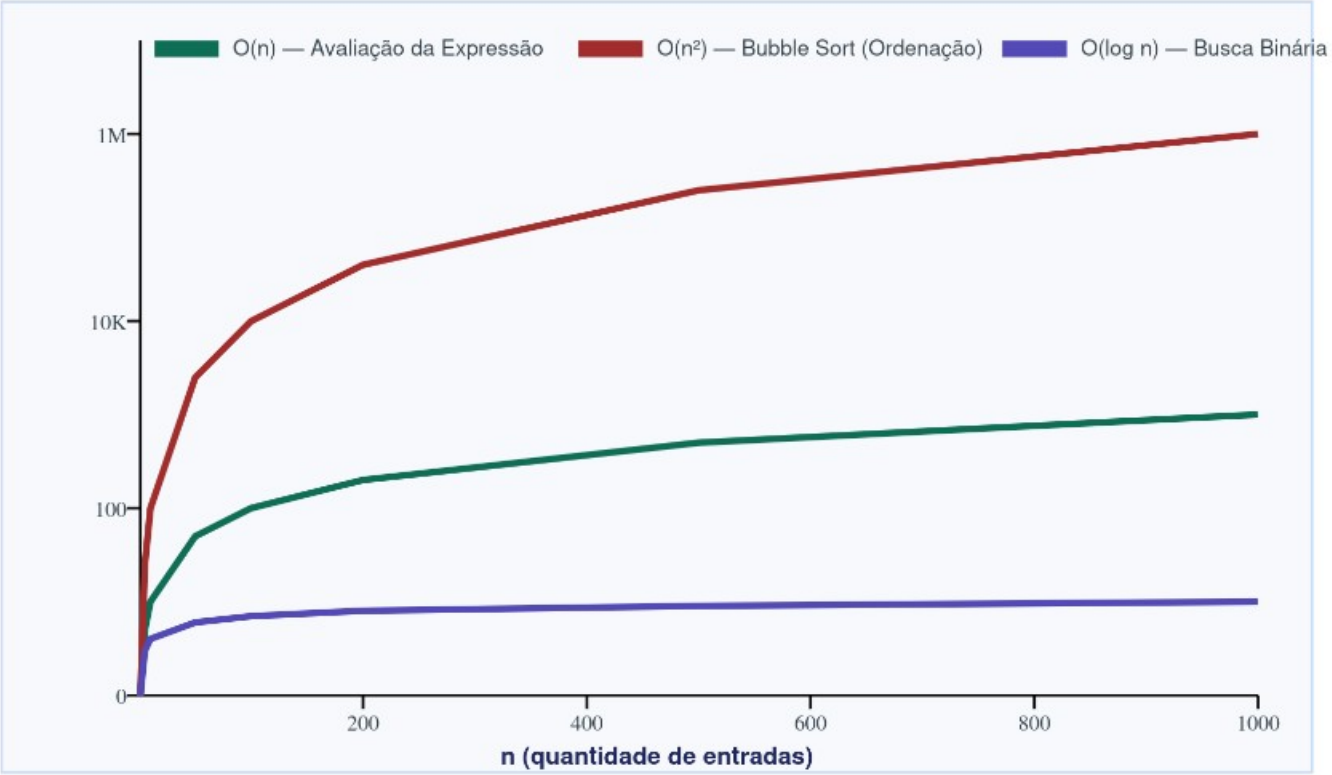
■ O(n) — cresce linearmente; muito eficiente

■ O(n^2) — cresce ao quadrado; gargalo do sistema

■ O(log n) — cresce logaritmicamente; excelente

Gráfico de Crescimento — Comparação Visual

Eixo X: n (quantidade de entradas). Eixo Y: número de operações (escala logarítmica).



Nota: O eixo Y está em escala logarítmica (log■■) para permitir visualizar as três curvas simultaneamente. Na escala real, $O(n^2)$ cresce de forma dramaticamente superior às demais.